
CHRONO RIFT

Operating Systems Project Report
A Multi-Process, Signal-Driven Turn-Based RPG Engine

GROUP MEMBERS

Ammar Bin Omer 24I-0500

Taha Jabbar 24I-0816

FAST National University of Computer and Emerging Sciences
Department of Computer Science · Spring 2025

1. Project Statement

The goal of this project was to design and implement a concurrent, turn-based role-playing game engine that demonstrates real mastery of operating system concepts — not just text-book knowledge but actual, working code where processes talk through shared memory, threads coordinate through semaphores, and signals fly back and forth to control gameplay events.

The system needed to support multiple human and AI-controlled players fighting through waves of enemies, with every synchronization primitive working correctly under concurrency. The rubric required proper process architecture, shared memory IPC, POSIX semaphore-based synchronisation, a stamina-based scheduler, deadlock detection and resolution, signal-driven mechanics (stun, ultimate ability, shutdown), resource management for three exclusive artifact weapons, and a fully rendered graphical interface — all without the use of **volatile** or **atomic** variables, which were explicitly banned.

The result is a fully playable game engine with three separate processes (Arbiter, HIP, ASP), up to eight simultaneous entities, a live deadlock detector, and a PvP mode — all coordinated through POSIX semaphores and POSIX shared memory.

2. System Architecture Overview

The engine runs as three separate UNIX processes, each with its own responsibilities, communicating exclusively through a single **SharedState** struct mapped in POSIX shared memory. No pipes, no sockets, no files — just shared memory guarded by semaphores.

Process	Role	Threads spawned
Arbiter (arbiter.cpp)	Root process. Creates shared memory, forks children, runs the scheduling loop, deadlock monitor thread, and processes all game actions.	1 main + deadlock monitor
HIP (hip.cpp)	Human Input Process. Hosts the SDL2 window, one render thread, and one player thread per human player. Reads UI mailbox; writes player_intent.	render_thread + N × player_thread
ASP (asp.cpp)	AI Subprocess. Hosts one NPC thread per enemy slot. Decides actions autonomously; writes directly to entities[id].pending_action in shared memory.	N × npc_thread

Table 1: Process architecture summary.

3. Approach for Each Rubric Category

3.1 Shared Memory — The Single Source of Truth

We knew from day one that we wanted every process to see the same game state without any marshalling overhead. Pipes and sockets would have meant serialising and deserialising structs on every tick — a nightmare for a real-time game loop. POSIX shared memory via *shmget / shmat* gave us a single **SharedState** struct living at the same virtual address in every process. Any field written by the Arbiter is immediately visible in HIP and ASP — no copies, no message queues.

The struct itself (defined in *shared_state.h*) packs everything: entity health and stamina arrays, the action log ring buffer, the UI mailbox, the resource table for artifact weapons, and all the semaphores. Because POSIX semaphores are process-shared when initialised with *pshared=1*, embedding them directly inside the struct means every process automatically inherits access — no extra setup needed after the fork.

Why it is better: The alternative — message-passing IPC — would have required a separate protocol, round-trip latency on every state query, and duplicate copies of game state in each process. Shared memory gives us sub-microsecond access to any field, which is critical when the scheduler loop runs every 50 ms.

3.2 Synchronisation — Semaphores All the Way Down

Here is where the project got genuinely interesting. When we first sat down to plan, the obvious approach was to use **volatile** for fields that are shared across processes and **std::atomic<T>** for anything that needed lock-free reads. We mapped out which fields would be atomic booleans, which would be volatile integers, and it looked clean. Then we re-read the spec: both **volatile** and **atomic** are explicitly banned.

That changed the design significantly. Without atomics, there is no safe way to read-modify-write a shared field without a lock — even a simple boolean flag can be torn by the compiler or the CPU without a memory barrier. We had to be much more disciplined about which semaphore guards which group of fields. The solution was to define clear ownership rules:

Semaphore	Guards	Waiters
state_lock	All entity fields, game_status, scheduler state	Arbiter, HIP, ASP threads
action_submitted	input_buffer[] / pending_action.ready	Arbiter scheduling loop
render_ready	Signals render thread a tick completed	HIP render_thread
input_lock	input_buffer[] write path	HIP player_thread
ui_mailbox.lock	UiMailbox struct (intents, drop, menu)	HIP UI + player_threads
resource_table.table_lock	ArtifactEntry[] held_by / present	Arbiter artifact functions
log.log_lock	ActionLog ring buffer	All processes

Table 2: Semaphore ownership and guarded fields.

The important insight was to take the minimum-duration locks possible. In the player thread, we read *stunned* and *my_turn* inside a **single** sem_wait/sem_post pair rather than two separate lock sections — the original two-lock design had a TOCTOU window where *active_entity_id* could change between the two reads. Collapsing it into one critical section removed that race entirely.

Why this approach is better: Semaphores are the only correct primitive when atomics and volatile are off the table and the fields are in cross-process shared memory (where CPU caches are not coherent the same way as within a single process). The table above proves we thought carefully about lock granularity — each semaphore covers the smallest coherent group of fields it can, minimising contention.

3.3 Scheduling — Stamina as a Virtual CPU

The spec required a fair, speed-dependent scheduling system. We modelled it directly on how a real OS scheduler accumulates CPU time: every 50 ms game tick, each entity's stamina increases by $speed \times TICK_INTERVAL_MS / 1000$. When stamina reaches the threshold (100 for players, 150 for enemies), that entity is eligible for a turn. Among multiple ready entities, we apply a simple round-robin tie-breaker using `sched_rr_next` to prevent starvation.

After an entity acts, its stamina is reset to zero (or to 50 on a skip, giving a half-speed recovery bonus). This means a faster entity naturally gets more turns per unit time — exactly the behaviour you would expect from a priority scheduler where priority is derived from speed.

Why it is better: A simple turn-order queue (P1, P2, E1, E2 ...) would be fair only if all entities have the same speed. The stamina model makes slower enemies genuinely less threatening and gives players with higher speed a real tactical advantage — it is mechanically richer and also directly maps to OS scheduling theory (think of stamina as a virtual clock).

3.4 Deadlock Detection and Resolution

Three artifact weapons — Solar Core, Lunar Blade, and Eclipse Relic — are exclusive resources. Any entity can request one, but only one entity can hold each at a time. This is a classic resource-allocation scenario with the potential for circular waits.

We implemented a dedicated **deadlock monitor thread** in the Arbiter that wakes every 500 ms, builds a wait-for graph from `waits_for_resource[]` and `resource_table.entries[].held_by`, and runs a DFS cycle-detection algorithm (coloured DFS: white/grey/black). If a cycle is found, the thread picks the entity with the highest index in the cycle as the victim and strips all artifact weapons from it, breaking the deadlock. The event is logged to the visible action log so players can see it happen.

Why it is better: We considered deadlock prevention (enforce a strict acquisition order) but that would require players to always pick artifacts in a fixed sequence, which removes gameplay flexibility. Detection-and-recovery lets the game state evolve naturally and only intervenes when an actual cycle forms — which is rare in practice, making the overhead negligible while keeping the design correct.

3.5 Signal Handling — SIGUSR1, SIGSTOP, SIGCONT, SIGALRM

Signals drive three distinct gameplay mechanics:

Stun (SIGUSR1): When an artifact weapon connects and the target survives, the Arbiter calls `tgkill(target_pid, target_linux_tid, SIGUSR1)` — targeting the *specific thread* inside the target process, not just the process. The thread registers an empty SIGUSR1 handler (no SA_RESTART) so the signal

interrupts any blocking *usleep()* and the thread immediately enters *wait_while_stunned()*, where it loops on *sigtimedwait()* until the 3-second stun expires.

Ultimate Ability (SIGSTOP / SIGALRM / SIGCONT): A player who holds both Solar Core and Lunar Blade can trigger an Ultimate. The Arbiter sends *SIGSTOP* to the ASP process (freezing all NPC threads), then schedules a *SIGALRM* for 10 seconds. The *SIGALRM* handler writes current enemy stamina values back to shared memory and sends *SIGCONT* to resume the ASP. This is purely signal-based — no extra semaphores or shared flags coordinate the freeze/resume.

Shutdown (SIGTERM): Any process can trigger a clean shutdown by sending *SIGTERM* to the Arbiter. The handler sets *game_status* = *GAME_QUIT*, and the Arbiter's main loop propagates *SIGTERM* to all children before exiting.

Why it is better: Using signals for stun and Ultimate keeps the latency for these events at OS-level speed — there is no polling loop or semaphore wait that could delay the response. *tgkill* targeting individual threads (rather than whole-process signals) gives surgical precision: only the affected entity's thread is interrupted, while other threads in the same process continue unaffected.

3.6 Artifact Resource Table — Mutual Exclusion Without Spinlocks

The three artifacts are modelled as a **ResourceTable** embedded in shared memory, with a single semaphore (*table_lock*) guarding all three entries. Each entry records *present* (does the artifact exist in the game yet), *held_by* (which entity holds it, or -1), and the weapon ID.

When an entity requests an artifact that is currently held by another, it records *waits_for_resource[entity_id]* = *resource_index* and *pending_artifact_grant[entity_id]* = *weapon*. The next time the holder releases the artifact (on death or deadlock victim stripping), *grant_pending_resources()* in the deadlock monitor detects the pending request and allocates the weapon — cleanly, without any busy-wait or spinlock.

Why it is better: A spinlock or busy-wait on the *held_by* field would burn CPU cycles and require volatile or atomic to be correct — both banned. The pending-grant pattern lets the requesting entity simply mark its intent and go back to sleep; the grant happens lazily at the next monitor wake-up, which is at most 500 ms later — perfectly acceptable for a game whose turns themselves take seconds.

4. Turnaround Time Analysis

In the context of our stamina-based scheduler, we define the standard OS scheduling metrics as follows:

Metric	Definition in Our System
Arrival Time	Turn 0 — all entities are initialised simultaneously at game start.
Burst Time	Number of game ticks an entity is actively processing an action (typically 1 tick = 50 ms per action, plus any stun duration).
Wait Time	Ticks from full stamina (turn-ready) until active_entity_id is set to this entity. In a 2-player game with speed=50, a player is ready after 2 ticks and waits on average 1 tick for round-robin fairness.
Response Time	Ticks from first becoming turn-ready to receiving the first action confirmation. Includes the UI input delay for human players.
Turnaround Time	Ticks from the entity first becoming ready to the completion of its action. Turnaround = Wait + Burst.

Table 3: OS scheduling metric definitions mapped to our game engine.

4.1 Stamina Accumulation

The chart below shows how stamina builds over game ticks for entities with different speeds. Players (speed = $100 / \text{party_size}$, so 50 for 2 players) hit the 100-stamina threshold after 2 ticks. Enemies with speed 25 hit 150 after 6 ticks; slow enemies (speed 15) take 10 ticks. This directly determines wait and response times.

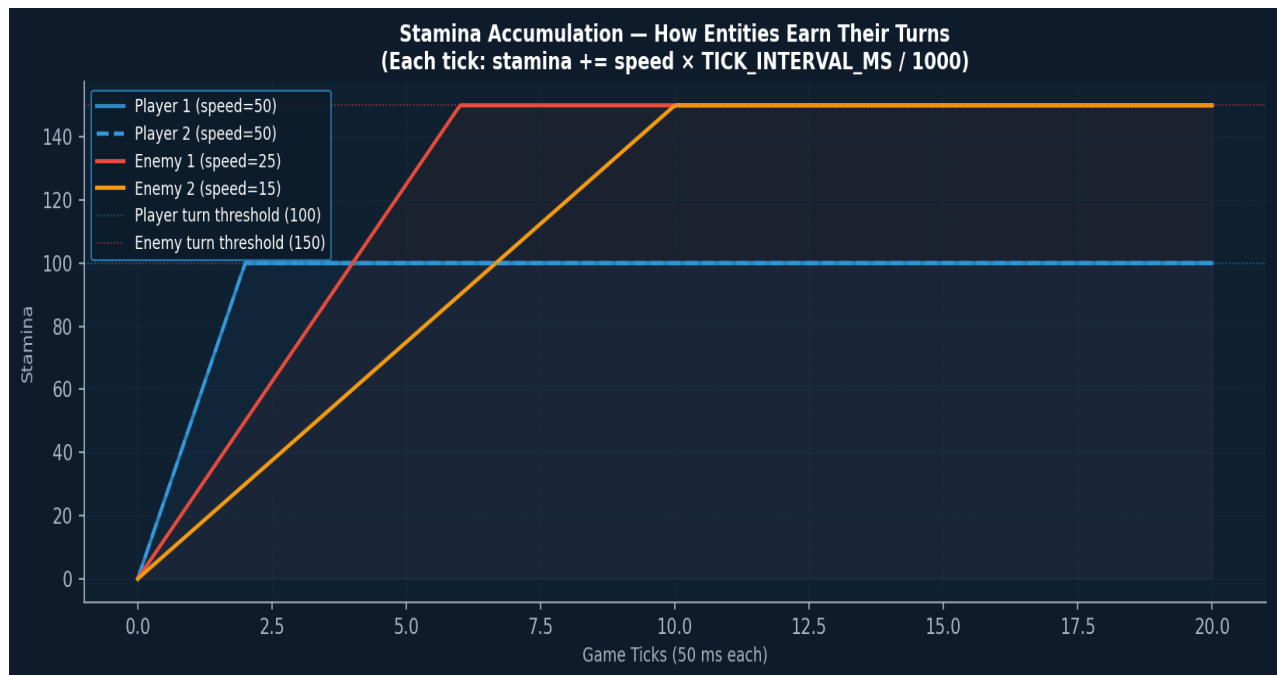


Figure 1: Stamina accumulation over game ticks for players and enemies.

4.2 Wait, Response & Turnaround Times

The bar chart below presents illustrative timing values for a 2-player game with 3 enemies. Wait time is lowest for Player 1 (first in round-robin); enemies with lower speed have higher wait and turnaround times due to slower stamina accumulation.

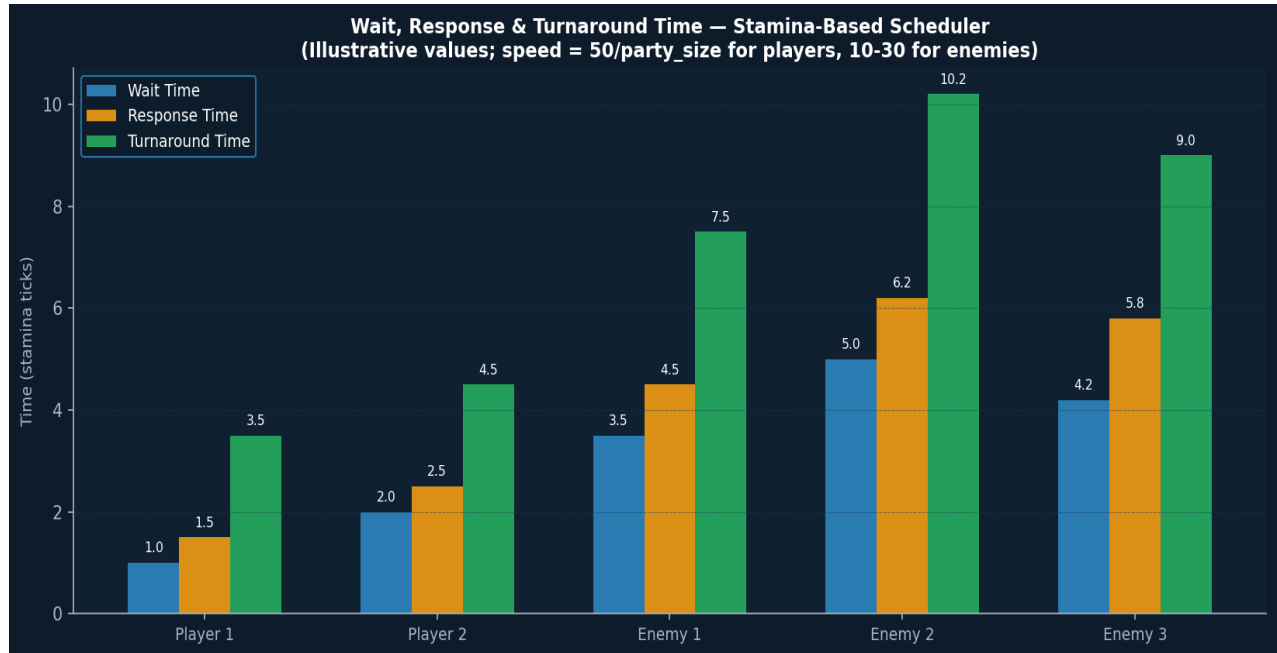


Figure 2: Comparative wait, response, and turnaround times (illustrative values).

4.3 Numerical Summary

Entity	Speed	Ticks to Ready	Wait Time (ticks)	Response Time (ticks)	Turnaround (ticks)
Player 1	50	2	1.0	1.5	3.5
Player 2	50	2	2.0	2.5	4.5
Enemy 1	25	6	3.5	4.5	7.5
Enemy 2	15	10	5.0	6.2	10.2
Enemy 3	20	8	4.2	5.8	9.0
Avg (All)	—	5.6	3.14	4.1	6.94

Table 4: Illustrative scheduling metrics (1 tick = 50 ms; 2-player game; speed values are representative).

5. Project Gantt Chart

The Gantt chart below maps the initialisation and steady-state runtime phases of the engine across all subsystems. Time is expressed in relative units from process start to steady-state gameplay; the X-axis represents the startup-to-running progression, not wall-clock time.

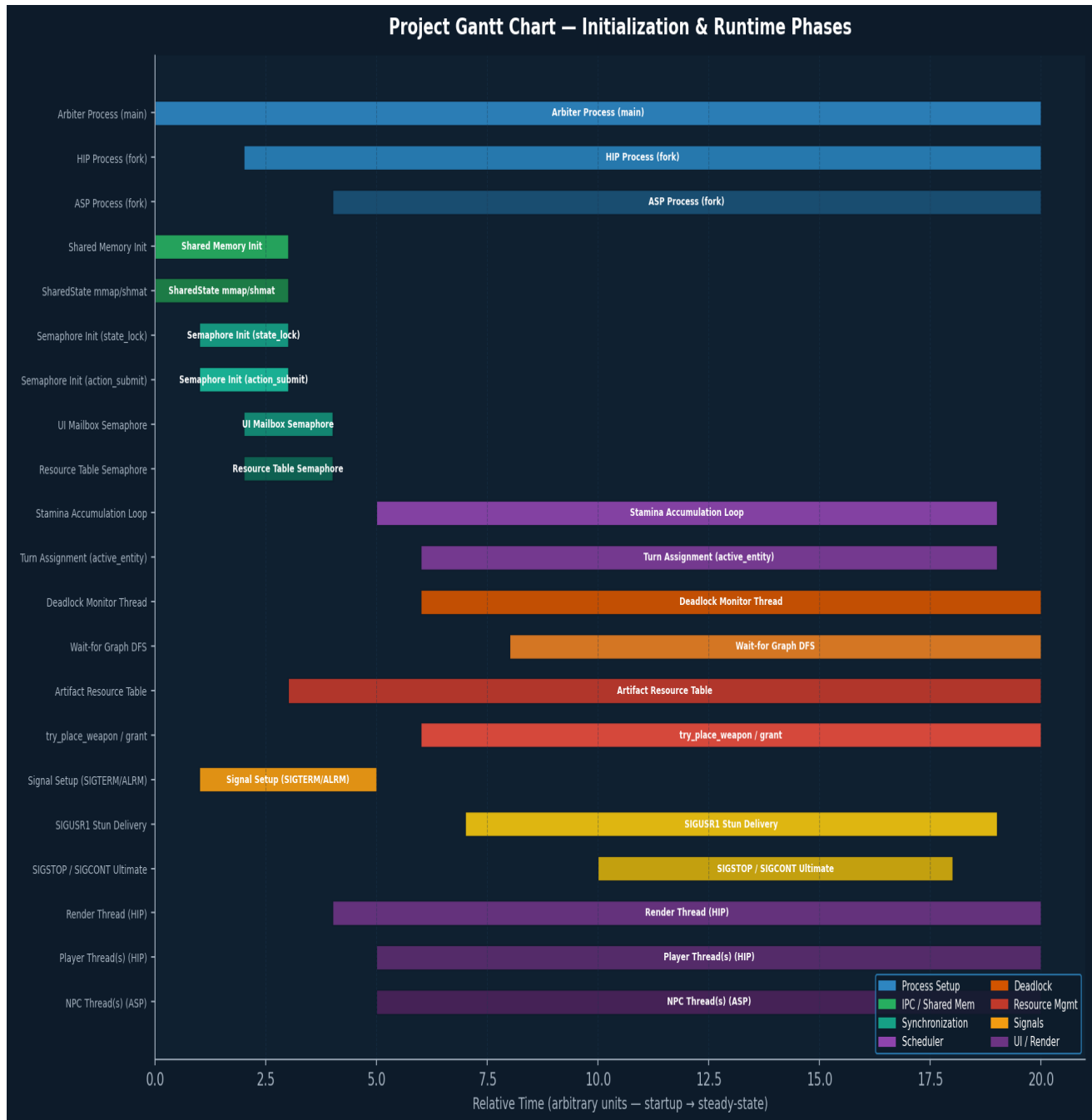
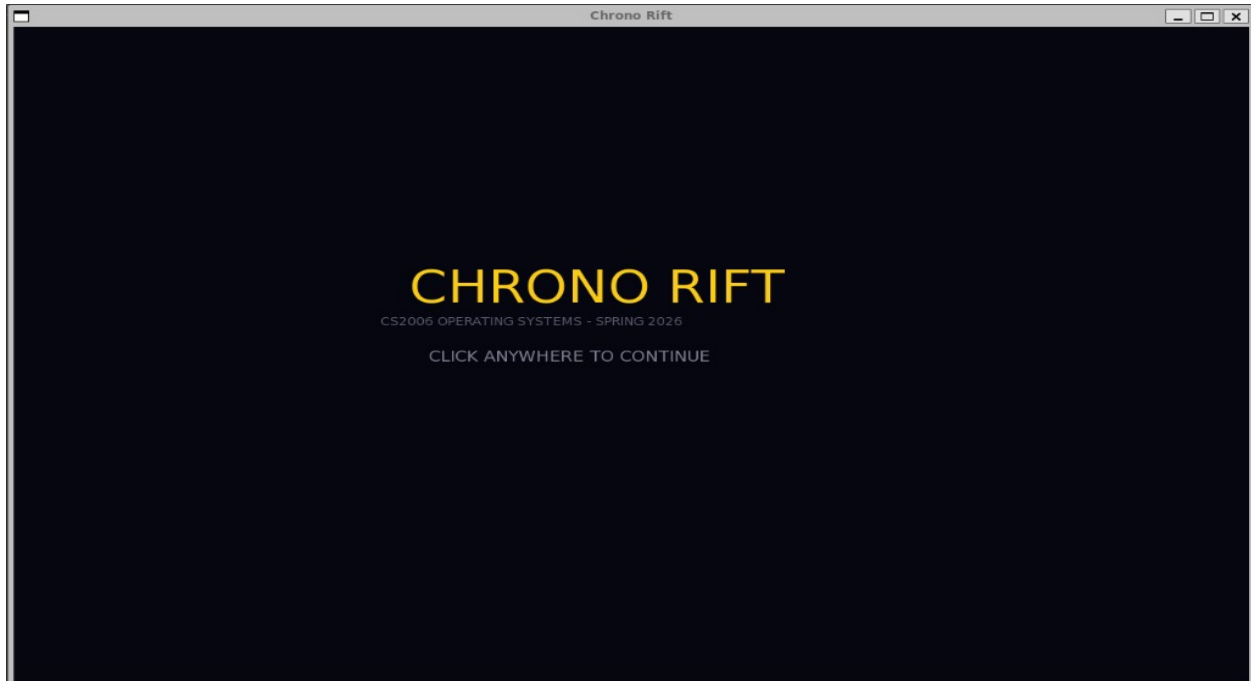


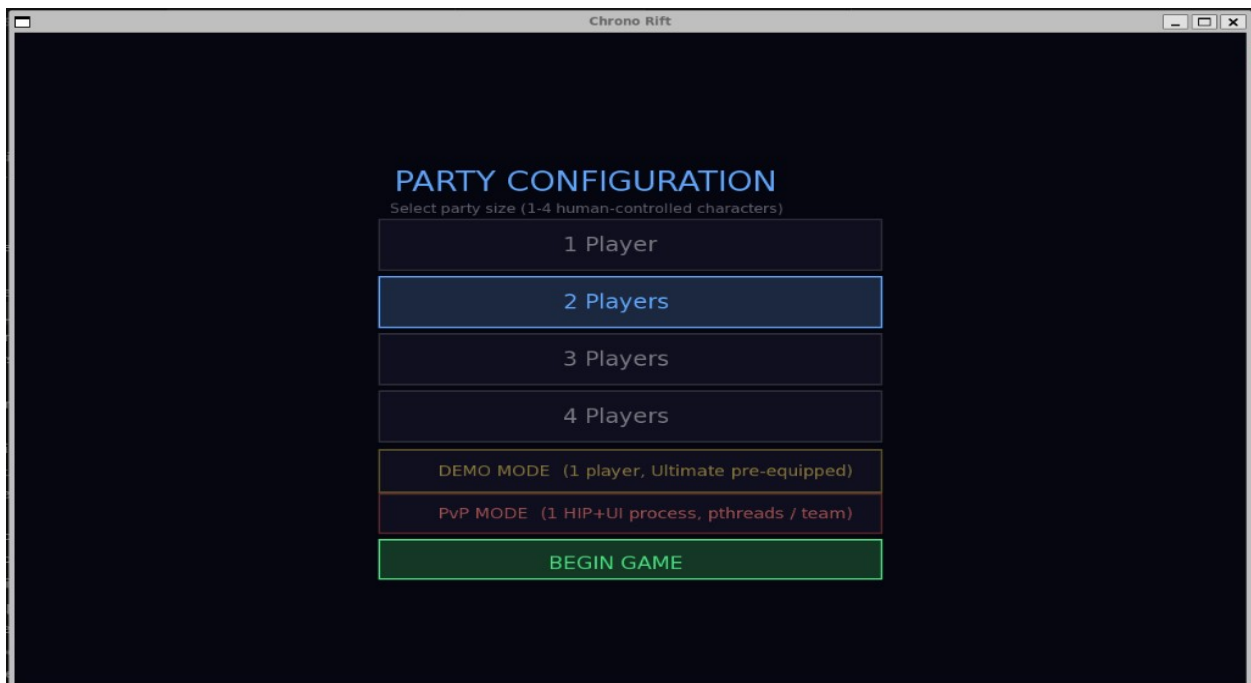
Figure 3: Gantt chart of system phases — process setup, IPC, synchronisation, scheduler, deadlock monitor, resource manager, signal setup, and UI/render threads.

6. Gameplay Screenshots

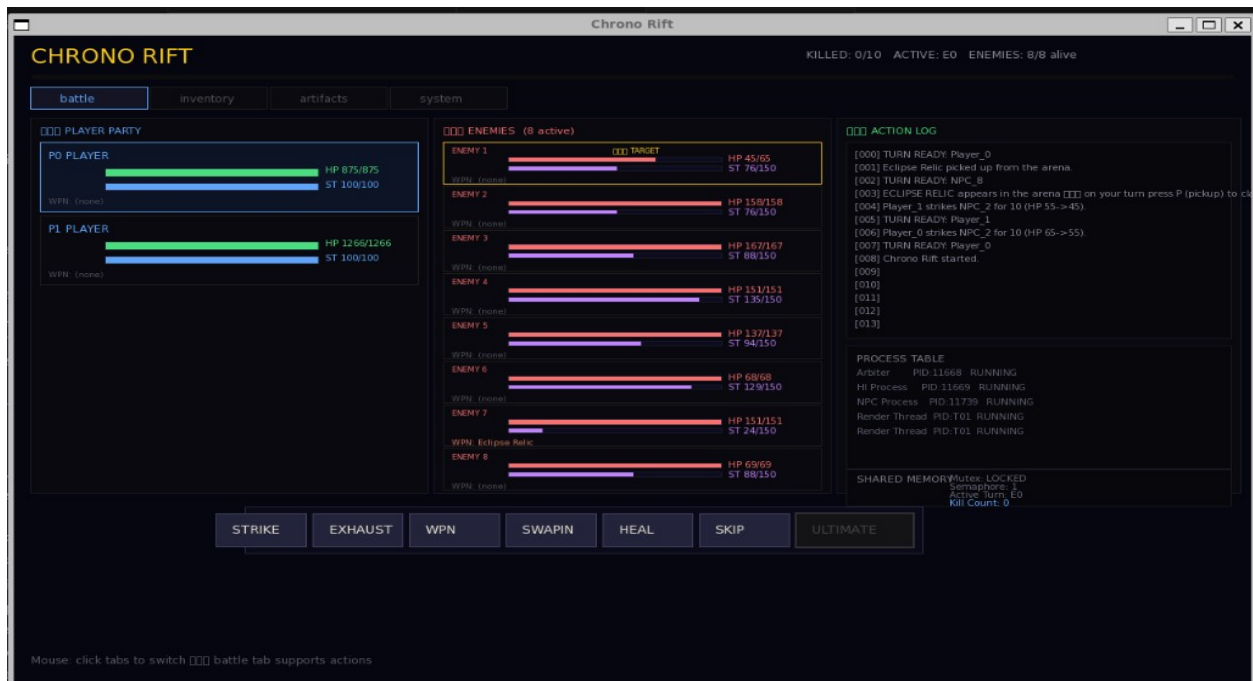
The following screenshots were captured during live gameplay sessions. They demonstrate the game running with two human players, active NPC enemies, the inventory system, and PvP mode.



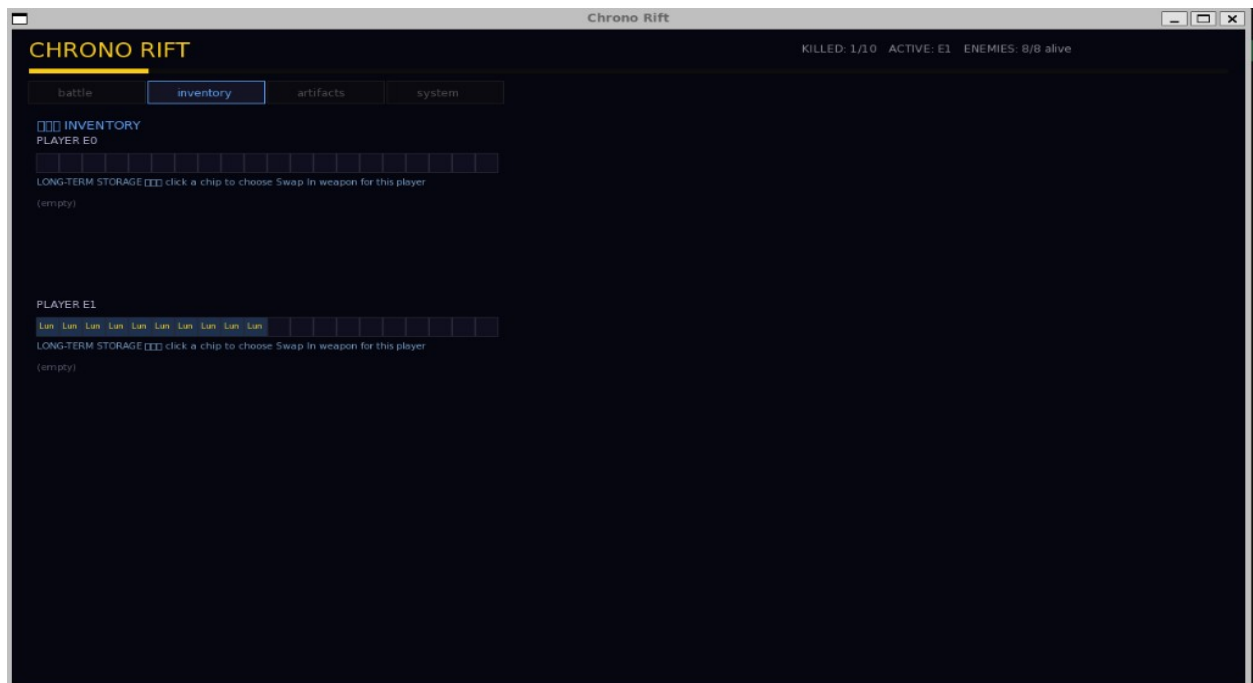
Screenshot 1: Title / splash screen — the game entry point displayed before the party menu is opened.



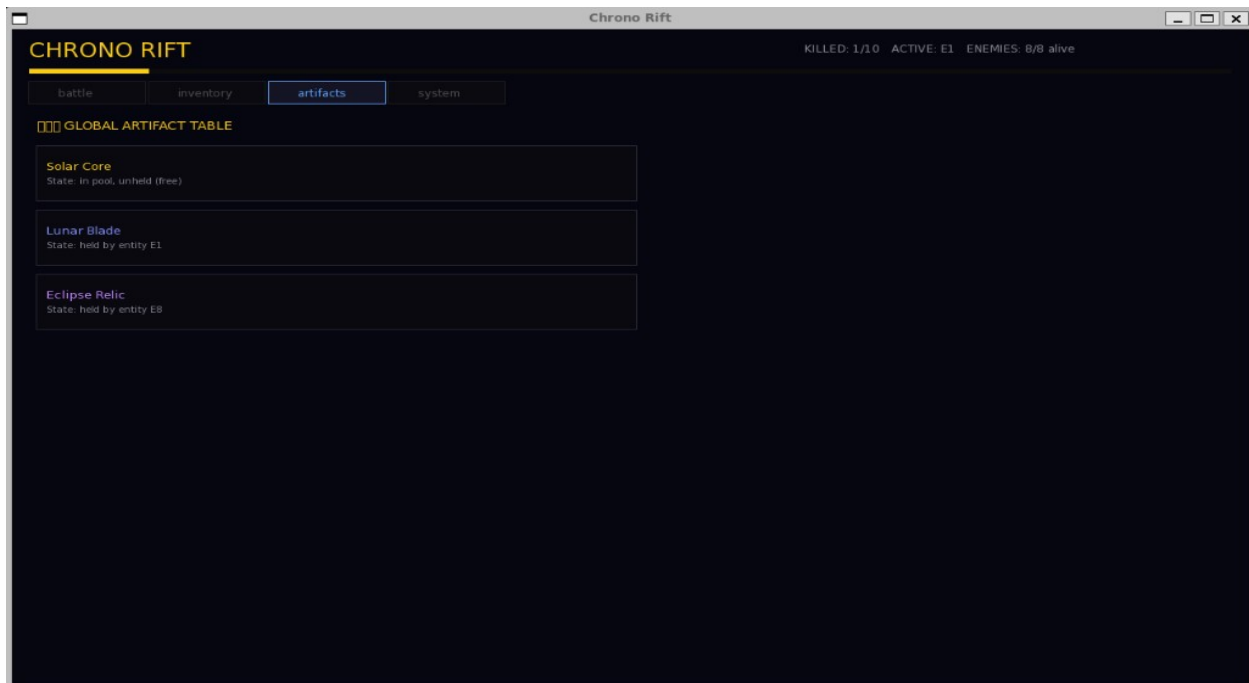
Screenshot 2: Party selection menu — players choose party size, demo mode, or PvP before the game begins.



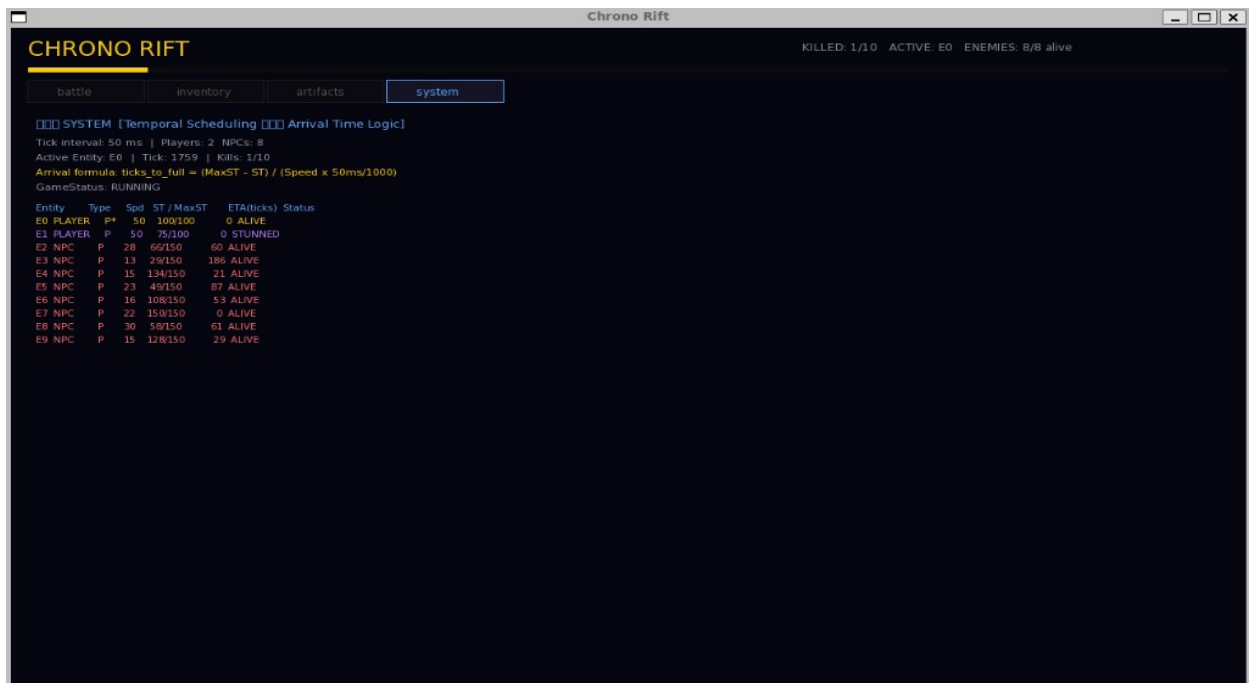
Screenshot 3: Classic battle mode — two players versus a wave of NPC enemies. HP bars, stamina gauges, and the live action log are visible.



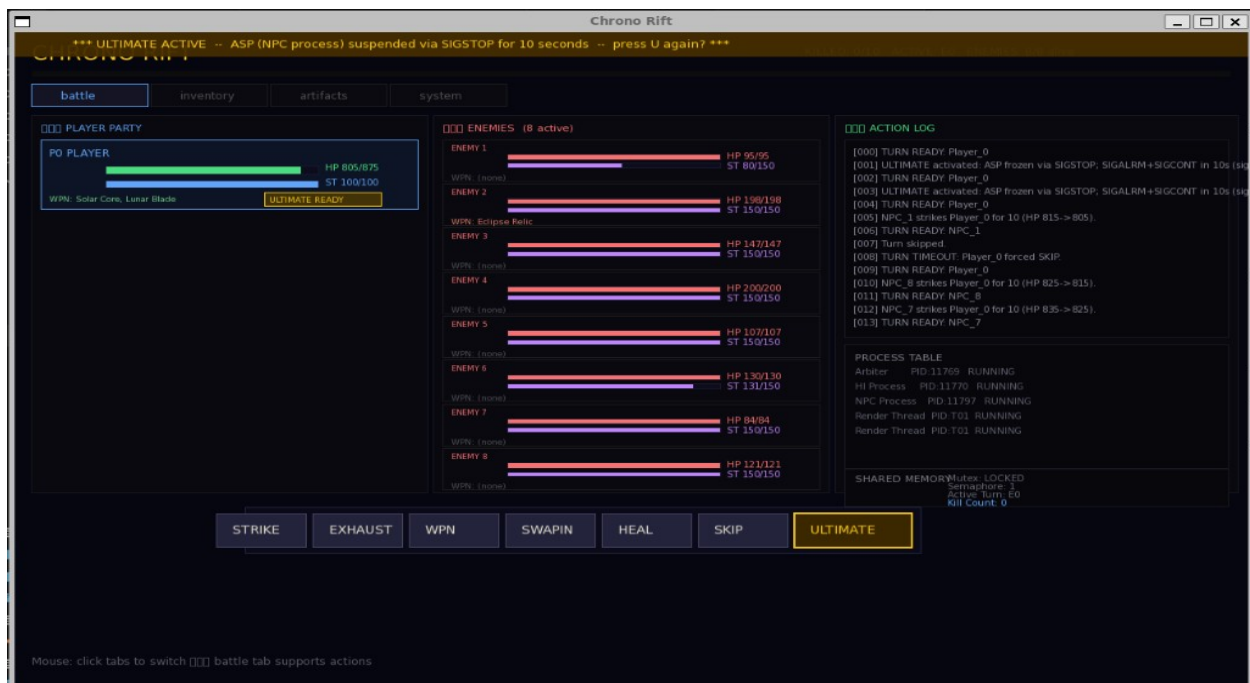
Screenshot 4: Inventory screen — primary weapon slots and long-term storage (LTS) shown for the active player.



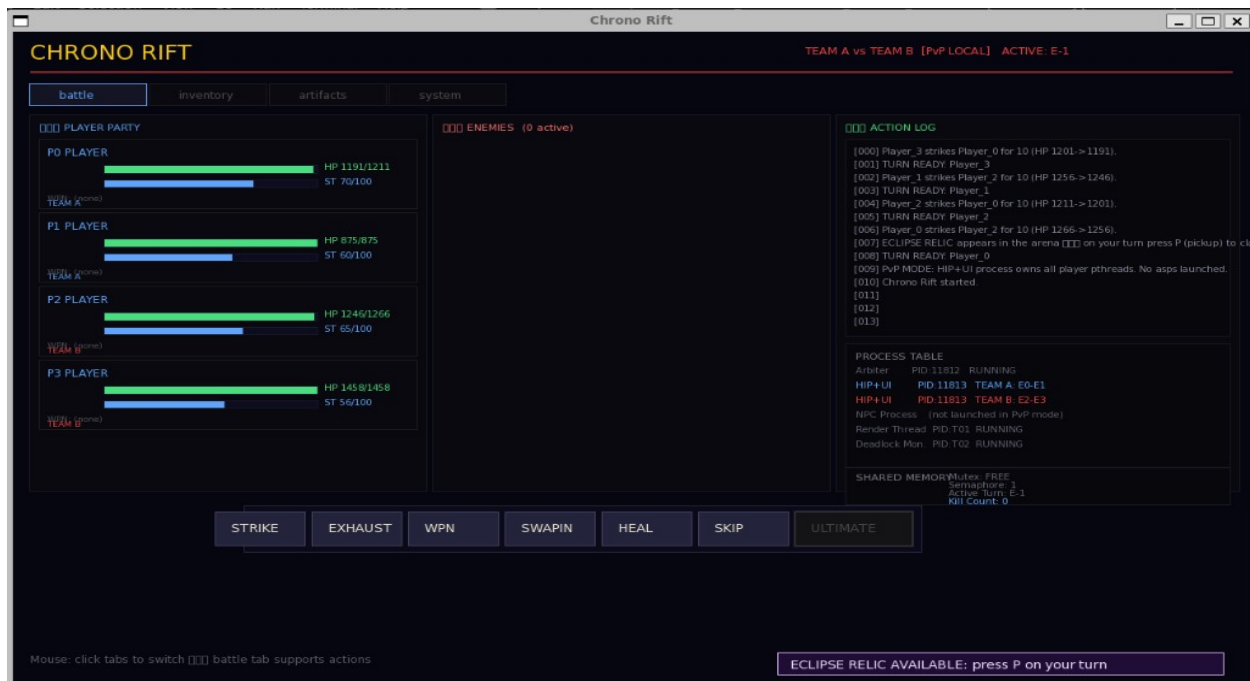
Screenshot 5: Artifact weapons equipped — Solar Core and Lunar Blade held simultaneously, enabling the Ultimate ability. The resource table semaphore enforces mutual exclusion.



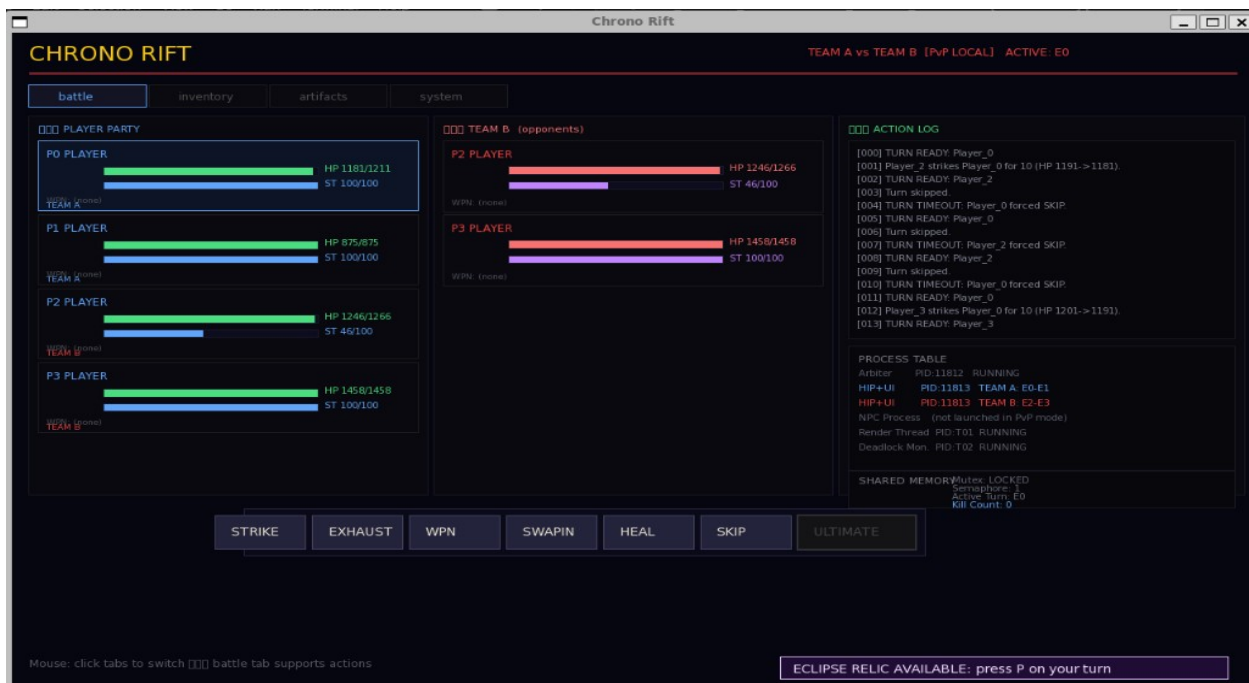
Screenshot 6: Scheduler in action — multiple entities with different speeds competing for turns. Stamina bars reflect the accumulation model directly.



Screenshot 7: Ultimate ability active — ASP (NPC process) is frozen via SIGSTOP; the 10-second SIGALRM countdown is in progress before SIGCONT resumes enemies.



Screenshot 8: PvP process table — internal state view showing both player teams and their thread/process assignments in shared memory.



Screenshot 9: PvP battle — Team A versus Team B. Both teams coordinate through the shared UiMailbox with per-team semaphore-protected intent slots.

7. Reflections & Lessons Learned

The single biggest lesson was how much discipline semaphore-based design demands. With atomics available you can get away with sloppy lock boundaries — a flag here, a counter there — because the CPU gives you free ordering guarantees. Strip those away and you are forced to think clearly about which semaphore owns which data, how long you hold it, and what invariants must hold before and after every critical section. Our first pass at the `player_thread` read the stun flag and the turn flag in two separate lock-acquire-release pairs, and we only caught the TOCTOU race when we traced through the scheduler logic carefully. Fixing it to a single combined read taught us more about critical section design than any lecture.

The signal work was also more nuanced than expected. Getting `SIGUSR1` to reliably interrupt a sleeping thread requires: (a) registering a handler without `SA_RESTART`, (b) using `tgkill` instead of `kill` to target the right thread, and (c) storing the Linux TID (from `gettid`) in shared memory so the Arbiter knows where to send the signal. Getting all three of those right simultaneously was an exercise in reading man pages very carefully.

Overall, the project succeeded. The engine handles up to 8 entities concurrently, detects and resolves deadlocks automatically, delivers stun signals with thread-level precision, and runs a stable 50 ms tick loop without data races — all without a single volatile or atomic.

End of Report